

Understanding Swarm Intelligence through Evolution-based Reinforcement Learning agents

Emmanuel Rassou

Harvard School of Engineering & Applied Sciences, 150 Western Ave.,
Allston, 02134, MA, United States.

Contributing authors: erassou@mit.edu;

Abstract

Understanding how collective intelligence arises in nature requires models that integrate learning, evolution, and social interaction. We extend a frontier multi-agent evolutionary engine with online reinforcement learning (RL) and minimal communication to study how these mechanisms shape population survival in ecological environments. Across food-growth conditions, adding RL to evolution improves mean survival time by up to 18.02%, demonstrating that even lightweight lifetime learning synergizes effectively with evolutionary selection. Introducing a localized communication channel yields an even larger improvement: survival time more than doubles, enabling populations to coordinate in stable, wave-like formations. Qualitative analyses reveal that communicating agents spontaneously recover hallmarks of swarm intelligence, including separation, alignment, and cohesion which are characteristic of natural flocks and herds. These results show that simple learning-evolution hybrids, when equipped with sparse social information, can reproduce complex animal-like collective behaviors. Our framework provides a foundation for scalable, biologically inspired studies of emergent social intelligence. Data and code open-sourced at https://github.com/emmanuel2406/dirt_rl.

Keywords: Reinforcement Learning at Scale, Evolutionary Simulation, Swarm Intelligence

1 Introduction

Online Reinforcement Learning (RL) offers a flexible framework for agents to explore and adapt within non-stationary environments, standing in contrast to dataset-centric supervised learning approaches. Its connections to biological systems have been widely studied over the past decade [1, 2]. While reward and incentive structures governing individual agents are often straightforward to interpret, incorporating social or cooperative dynamics within a population introduces richer, more biologically grounded phenomena. These dynamics hint at the possibility that RL systems can display early forms of cultural accumulation as they evolve. As demonstrated in JaxLife, communication mechanisms further enrich the behavioral ecology of multi-agent systems [3].

Even simple agents equipped with stationary, lifetime-fixed policies can exhibit complex group-level behavior when placed in ecological contexts [3, 4]. Building on this foundation, we extend the NomNom environment from the Distributed Intelligent Replicator Toolkit (DIRT) [5]. In such settings, the number of independently simulated agents becomes a central determinant of whether emergent population-level behaviors can be observed. Achieving scale, however, poses a fundamental tradeoff: maintaining simplicity in agent design is often necessary to preserve stability during training, especially when compared to more expressive RL algorithms that aim to learn an environment fully with a single agent.

To address this, we compress the environment to enable simulation of up to 128 concurrent agents on a single A100 GPU as seen in Figure 2. This paper examines ablations of this experimental setup, with a particular emphasis on interpreting the resulting behaviors through the lens of evolutionary dynamics and swarm intelligence.

Lastly, we consider extending the agent observations with information-sharing between agents, enabling the emergence of social behavior. Although this mechanism is implemented using an oracle, it provides a useful foundation for future work aimed at enriching the action space with explicit social cues.

Specifically, our contributions are:

- **Dirt-RL**: an extension of a state-of-the-art evolutionary engine for lightweight RL agents;
- performance boost on top of pure evolution by adding stable RL updates and communication;
- recovery of classical swarm intelligence algorithms through agent-agent communication.

2 Formulation

2.1 Environment: NomNom

Each agent can exist at any position (x,y) on a 512x512 pixel grid. Each cell is either (1) empty, (2) contains food, (3) contains an agent or (4) out-of-bounds. The food grid is simulated by $\text{Growth}(\lambda^{food}, \max^{food})$. The growth of food follows a Poisson distribution with mean $\lambda^{food} > 0$ and the simulator returns $G \in \{0, 1\}^{512 \times 512}$ where $G_{ij} = 1$ when a food spawns on that cell, and this is capped by $\max^{food}$ which represents the maximum number of new spawn locations to add.

The observation space is defined by two components:

- (v_w, v_d) where v_w and v_d are view width and view depth respectively.
- the current energy e of the agent

This is then vectorized per agent with extra dimension *max_players*.

Every agent’s life is determined by its energy e which is initialized at e^{init} and capped at e^{max} , such that it dies when $e \leq 0$. We have metabolism parameters as $m^{food}, m^{wait}, m^{move}, m^{reproduce}$.

For mutations, Gaussian noise is added at every step with $\Delta \sim N(0, lr^{mutate^2})$. We set $lr^{mutate} := 3 \times 10^{-4}$ which gives the update rule:

$$\theta_{new} := \theta_{old} + \Delta$$

2.2 Action-based simulation

The discrete action space is formulated as $A := A_1 \times A_2 \times A_3$

- A1 **forward**: $moved = 1$
- A2 **rotate left or right**: $moved = 1$
- A3 **reproduce**: only if $e > e^{init}$ then $reproduce = 1$

Thus, with $|A| = 12$ the space is tractable for linear models. The energy is then updated per time step as:

$$e = e - moved \times m^{move} - (1 - moved) \times m^{wait} - reproduce \times e^{init}$$

The goal for each agent is to survive as long as possible, and this goal is carried over to the reproduced agents.

Disclaimer: there are other environment parameters (e.g. senescence) that have been set to the default from the original DIRT repo [5]. These may be further tuned in future work. For the purposes of this paper, the environment is stable enough to yield results for suboptimal configurations.

2.3 Policy Modeling

Given that the lifetime of each agent is smaller than a conventional RL episode, DIRT considers simpler models of the policy. They use three parallel linear decoders – one

for each action set – with each taking the same dimensional input to give 539 trainable parameters.

3 Reinforcement Learning

Our main contribution extends the evolution-setup of DIRT to allow for an agent’s policy to be updated during its lifetime. These updates happen online with Reinforcement Learning. The agents now learn within their lifetime, with evolution now self-selecting for the best learners.

3.1 Individual RL Adaptation

We implement on-policy policy gradient updates at regular intervals.

Batch updates. We have a buffer with size $T = 5$. This does not function the same as the replay buffer in off-policy methods [6]. Once the buffer is full, a policy gradient is performed on the whole batch of experiences and their collected rewards. This reduces the variance of updates which is essential for stable learning in a non-stationary environment. Subsequently the buffer is reset to mitigate the effect of stale experiences, and the process repeats. Our setup encourages small buffer sizes so faster adaptation can occur from more frequent online updates, given the risk of agents dying too early.

Return and Rewards. We use a simple reward shaping mechanism which incentivizes both staying alive with weight $w^{survival} = 1.0$ and eating food $w^{food} = 0.1$. It is possible to have more intricate reward functions, but for a proof of concept that the multi-agent system learns in *NomNom*, this simple formula should suffice.

$$r_t = (e_{t+1} - e_t)w^{food} + w^{survival}$$

We have a discount factor $\gamma = 0.95$. The discounted return at local timestep t within the buffer is:

$$R_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$$

Advantage Normalization. To reduce variance even more and ensure stability numerically and with respect to different learning rates, we subtract the baseline as the average return which represents an estimate of the value of that state and calculates the advantage.

$$A_t = \frac{R_t - \frac{1}{T} \sum_{t=0}^{T-1} R_t}{\sigma_R}$$

where σ_R represents the variance calculation.

Policy Gradient Update. For a batch of experiences $\{o_t, a_t, A_t\}_{t=0}^{T-1}$ we calculate the loss as:

$$L(\theta) = -\frac{1}{T} \sum_{t=0}^{T-1} A_t \cdot \log \pi_{\theta}(a_t|o_t)$$

$$\theta_{new} := \theta_{old} - \alpha \cdot \nabla_{\theta} L$$

3.2 Inter-Agent Communication

We augment each agent’s observation space with oracle information from nearby agents. Specifically, the observation includes:

- *max_neighbors*: the maximum number of agents to track within local radius
- $\mathbf{r}_i = (dx_i, dy_i)$: the relative position of neighbor i
- e_i : the energy level of neighbor i
- d_i : the Euclidean distance to neighbor i

In future work, we plan to extend the action space with an additional **message** action, enabling agents to learn explicit bidirectional communication. This removes the need for oracle-based information sharing that leverages spatial locality as a proxy for communication intent.

4 Experimental Results

4.1 Experimental Setup

To save memory costs and avoid Out-Of-Memory failures on a limited GPU setup, we store minimal information to reconstruct a pixelized 2D grid as shown in 2.

Additionally, these food grids are checkpointed every `STEP_STRIDE` timesteps and then synthesized to a video with `ffmpeg`. An example video of one of the simulations with step size 1 is shown [here](#)

We use `jax`’s random key function to set seeds from $1234 \rightarrow 1234 + n - 1$ where n is the number of tries. The mean of the results is then reported to ensure robust findings across differences in randomization. Typically, we set $n = 5$ as per the convention in previous literature.

We are also able to run at least 3 experiments in parallel on a single A100. This is set by the `PARALLELISM` field with its own custom thread pool for experiments.

4.2 Comparing RL with Evolution

As a preliminary experiment to validate our extension of the policy to have Reinforcement Learning updates, we do an ablation on the *mean food growth* of the environment, keeping all other environment parameters at the default. To save compute we do a maximum of 20 epochs with 1024 steps per epoch.

The mean performance across five random seeds (1234, 1235, 1236, 1237, 1238) is reported in Table 1 for food-growth rates $\lambda^{\text{food}} \in \{1, 4, 16\}$. As expected, the survival time of the population increases with higher λ^{food} . Across all settings, our RL extension consistently outperforms the evolution-only baseline, achieving up to an 18.02% improvement at $\lambda^{\text{food}} = 4$. These results indicate that even simple linear policies can effectively leverage experience from a single agent, and that these updates synergize well with evolutionary self-selection. We emphasize that the RL gains reported here represent a lower bound: with more careful hyperparameter tuning (e.g., learning rate 10^{-4} , buffer size $T = 5$, discount factor $\gamma = 0.95$), the performance gap between the baseline and our method could likely be further increased.

More detailed results are shown in Figure 3 for just evolution and Figure 4 for evolution + RL. In particular 5 shows that the gains by our method are intentionally attributed to maximizing the discounted return since the average agent reward per step increases given more time to learn.

Table 1: Mean time of survival for different mean food growths

λ^{food}	1	4	16
Evolution	5447.00,	6640.60,	14219.20
Evolution + RL	6295.40	7837.20	14315.00
Performance gap Δ	+848.40(15.58%)	+1196.60 (18.02 %)	+95.80 (6.74 %)

There are many other points of comparison possible which we show as additional material in Appendix C. For example, we plot results for max energy $\in \{4, 8, 12\}$ in Figure 8.

4.3 RL with Communication enabled

As discussed in Section 3.2, we augment the observation space to include metadata about nearby agents. We set `communication_radius=5` to be the zone of grid cells to detect neighbors and `max_neighbors=8` as the maximum neighbors that can be observed to avoid memory problems.

Our experiments compare the base RL method with the augmented method of RL + Comm. Firstly, we ablate over the view distance $v_d \in \{1, 5, 15\}$ in Figure 6. For RL only, we see that $v_d = 1$ performs the best by a large margin with the population surviving **13.9k** timesteps. This makes sense since the agents do not need to detect food too far ahead of them, and expanding the dimensions of the linear model will only make the policy harder to learn. On the other hand for RL + Comm, $v_d = 5$ performs the best with the population surviving **34.4k** timesteps which is 147% more than the RL only case. A higher view distance is beneficial for better detecting nearby agents.

Figure 7 shows similar results by ablating wait metabolism $m^{wait} \in \{-0.1, -0.025, 0\}$ with $m^{wait} = -0.1$ performing the best for RL only with **13.1k** timesteps and also RL + Comm with **34.3k** timesteps to show if nature penalizes agents that stand still, they can learn more robust policies to survive. Of course, this huge performance boost could be reduced to the increase in information available; however, as we will see in the Discussion section, it is also due to emergent behaviors.

5 Discussion

If we consider a qualitative analysis of the longest surviving multi-agent systems that use communication, we find consistent wave-like behavior as shown in Figure 1

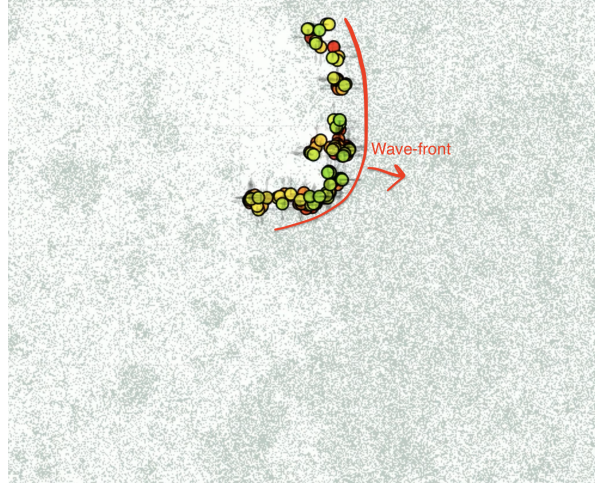


Fig. 1: Visualization of view distance $v_d = 5$ experiment with RL + comm.

This clumping of agents into a coordinated flock recovers *Boids flocking algorithm* [7]. The emergent behavior aligns strongly with its three foundational principles:

1. *Separation*: Steer to avoid crowding or colliding with very close flockmates. Each boid maintains a minimum distance from others within its "protected range, which avoids being too dense in food-sparse areas
2. *Alignment*: Steer towards the average heading (velocity and direction) of local flockmates. This encourages collective movement toward areas of higher resource availability while avoiding depleted regions behind the wave.
3. *Cohesion*: Steer to move towards the average position (center of mass) of nearby flockmates, which enables the wave to retain a coherent structure typically guided by a subset of "leaders" at its center.

Recovering these principles demonstrates that reinforcement learning is capable of reproducing hallmark emergent behaviors characteristic of evolutionary and biological swarm systems. Future work may extend the expressiveness and bandwidth of agent communication, enabling the emergence of more intricate, socially coordinated behaviors akin to those observed in complex civilizations.

6 Conclusion

We extended a frontier multi-agent evolution engine by integrating both reinforcement learning (RL) capabilities and a primitive communication channel. This hybridization of evolution and RL consistently improved population survival in the NomNom environment, demonstrating the utility of combining these paradigms. By examining spatiotemporal patterns in agent behavior, we also recovered signatures of the classic Boids flocking algorithm when communication was enable, which is an emergent phenomenon long associated with swarm intelligence systems.

We hope this codebase and these preliminary findings serve as a foundation for future explorations at the intersection of evolution, learning, and communication; not only to surpass established benchmarks, but to illuminate how complex collective behaviors observed in nature might arise from simple computational principles.

References

- [1] Neftci, E.O., Averbek, B.B.: Reinforcement learning in artificial and biological systems. *Nature Machine Intelligence* **1**, 133–143 (2019) <https://doi.org/10.1038/s42256-019-0025-4>
- [2] Muller, T.H., Butler, J.L., Veselic, S., Miranda, B., Wallis, J.D., Dayan, P., Behrens, T.E.J., Kurth-Nelson, Z., Kennerley, S.W.: Distributional reinforcement learning in prefrontal cortex. *Nature Neuroscience* **27**, 403–408 (2024) <https://doi.org/10.1038/s41593-023-01535-w>
- [3] Lu, C., Beukman, M., Matthews, M., Foerster, J.: JaxLife: An Open-Ended Agentic Simulator (2024). <https://arxiv.org/abs/2409.00853>
- [4] Bejjani, J., Amburg, C.V., Wang, C., Su, C.H., Pratt, S.M., Mazloumi, Y., Khoshnevis, N., Kakade, S.M., Brantley, K., Walsman, A.: The Emergence of Complex Behavior in Large-Scale Ecological Environments (2025). <https://arxiv.org/abs/2510.18221>
- [5] aaronwalsman: dirt. <https://github.com/aaronwalsman/dirt> (2024)
- [6] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D.: Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015) <https://doi.org/10.1038/nature14236>
- [7] Reynolds, C.W.: Flocks, herds and schools: A distributed behavioral model. In: *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '87)*, vol. 21, pp. 25–34. ACM, ??? (1987)

A RL Results

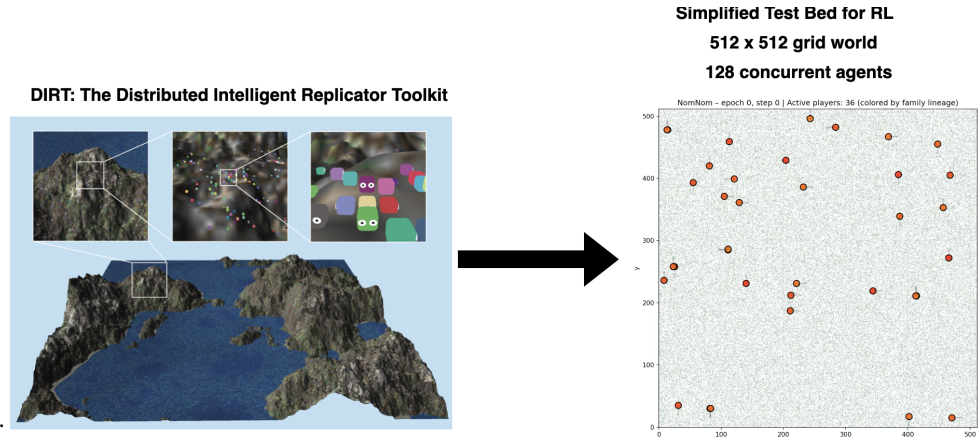
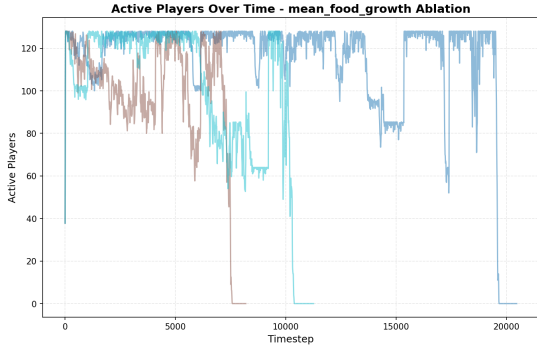
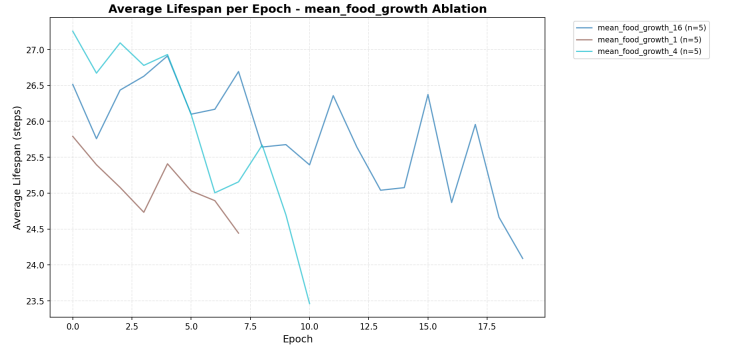


Fig. 2: Simplifying the visualization process for DIRT

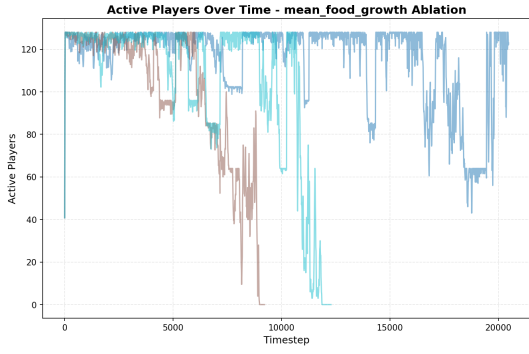


(a) Player count over time

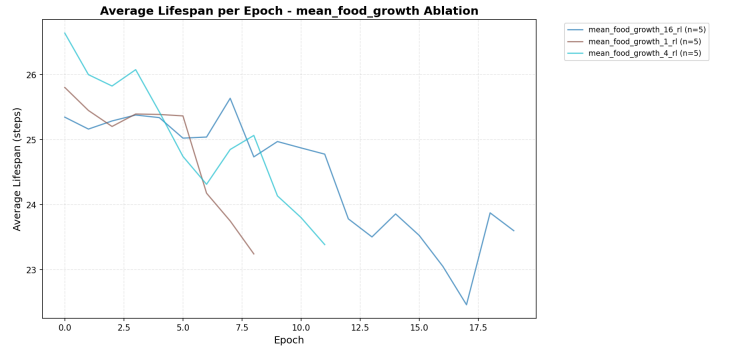


(b) Average life span over time

Fig. 3: baseline:- Mean food growth ablation with just evolution over 5 seeded tries



(a) Player count over time



(b) Average life span over time

Fig. 4: our method:- Mean food growth ablation for evolution + RL over 5 seeded tries

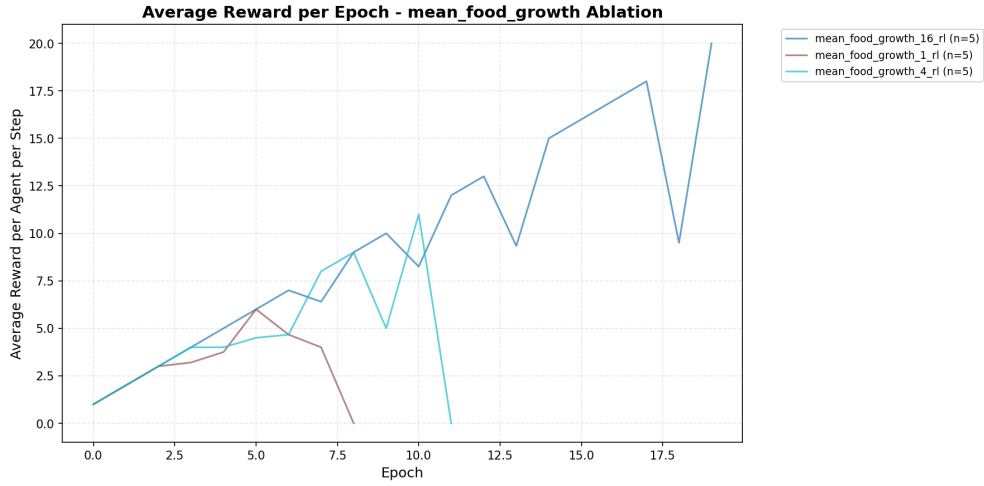
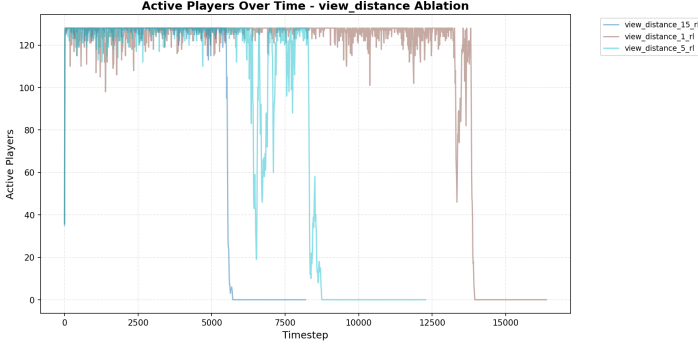
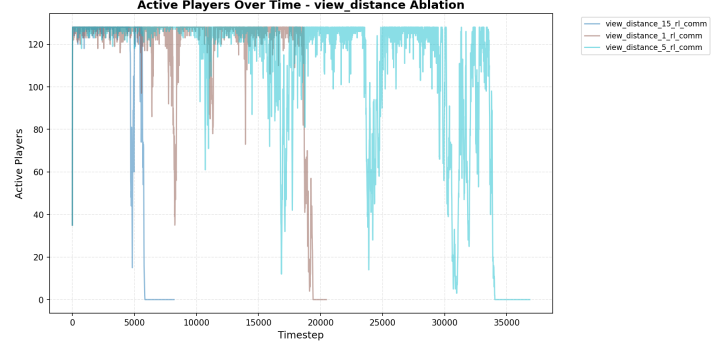


Fig. 5: Average agent reward per step for Evolution + RL ablating mean food growth

B Communication Results

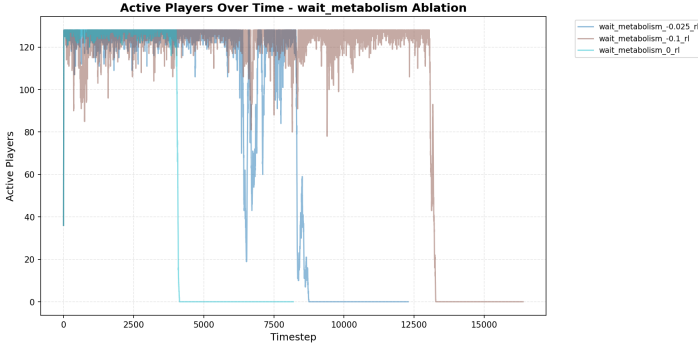


(a) Player count over time for RL

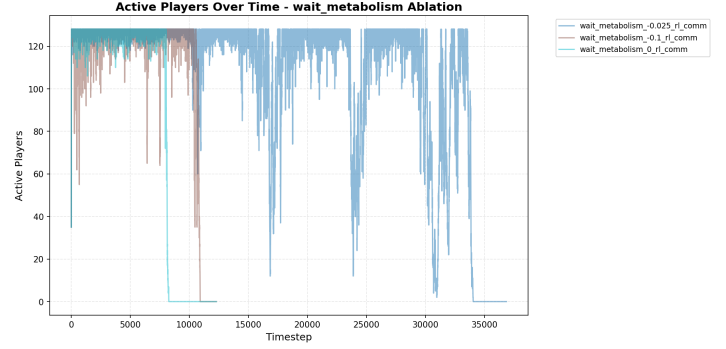


(b) Player count over time for RL + comm.

Fig. 6: Ablating view distance with and without communication enabled



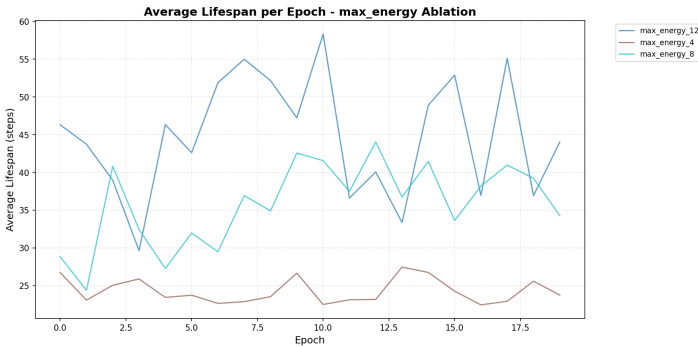
(a) Player count over time for RL



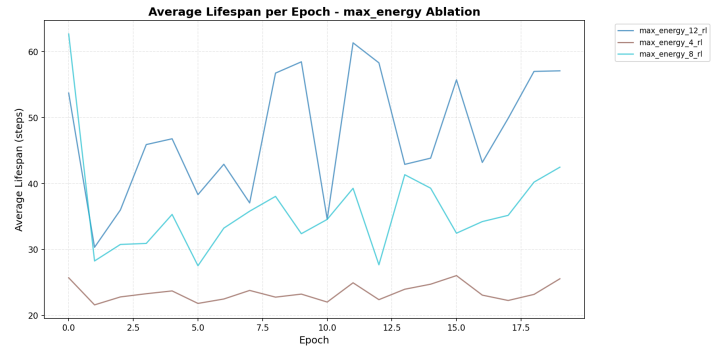
(b) Player count over time for RL + comm.

Fig. 7: Ablating wait metabolism with and without communication enabled

C Additional Ablations

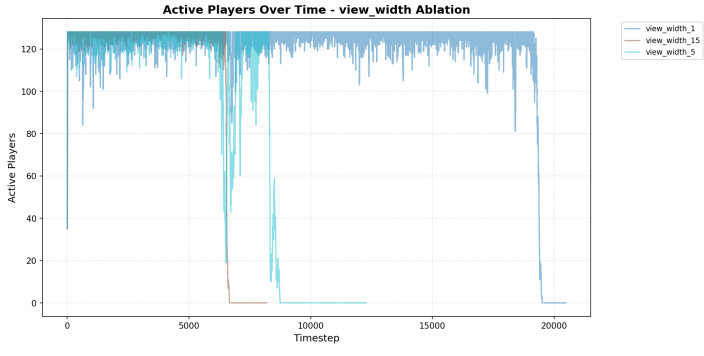


(a) Average lifespan of an agent with pure evolution

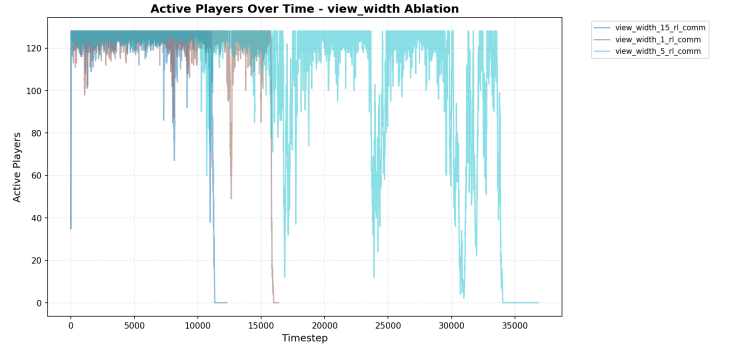


(b) Average lifespan of an agent with evolution + RL

Fig. 8: Ablating max energy with and without RL



(a) Player count over time with pure evolution



(b) Player count over time with evolution + RL

Fig. 9: Ablating view width with RL + communication and without RL